TECHNICAL REPORT · WAVEFORMJAX

IMRPhenomXPNR and NRSur7dq4 in JAX, checked against LALSimulation

How two precessing waveform models, and LALSimulation's conversions between the time and frequency domains, were written as pure JAX functions that compile, vectorise and differentiate. The report covers how they compare with LALSimulation 6.2.0 and with waveformrs, how their derivatives compare with finite differences, and what they cost.

REPOSITORY
`waveformjax`, branch `main` at `fe9389f` (86 commits)

REFERENCE
lalsimulation 6.2.0, lal 7.7.0 (conda-forge, osx-arm64), and waveformrs's Python bindings

PLATFORM
Apple M2 Ultra, 24 cores; JAX 0.11.2 on the CPU; Python 3.12.15

GENERATED
24 October 2026

CONTENTS

1 Summary
2 Models as pure JAX functions
3 IMRPhenomXPNR
4 NRSur7dq4
5 Time and frequency domains
6 How the package is tested
7 Accuracy
8 Derivatives
9 Performance
10 Differences from LALSimulation
11 Limitations and future work
12 Reproducing the results
13 Commit history

1 Summary

waveformjax implements the frequency-domain model IMRPhenomXPNR and the time-domain surrogate NRSur7dq4 in JAX, with the interface of waveformrs's Python bindings and LALSimulation's conventions. Every model is a pure function of a `Binary` pytree, so it can be compiled with `jax.jit`, evaluated for many binaries at once with `jax.vmap`, and differentiated with respect to all twelve parameters of the binary with `jax.grad`, `jax.jvp` and `jax.jacfwd`. LALSimulation's `SimInspiralTD` and `SimInspiralFD` are reproduced as well, with LAL's lengths and start times, and are differentiable too. The models follow LAL's code branch for branch, with each branch written as traced arithmetic. They agree with LALSimulation 6.2.0 to rounding for NRSur7dq4, and for IMRPhenomXPNR to the level at which different builds of LAL agree with each other.

Open items. The tests, the accuracy comparisons, the derivative checks and the timings below are complete. Still open:

- why finite differences of the full IMRPhenomXPNR waveform disagree with its derivatives in the in-plane spins (the derivatives section)
- JAX on one core: setting XLA's thread flags (`--xla_cpu_multi_thread_eigen=false intra_op_parallelism_threads=1`) did not change the ~3 cores it keeps busy, so the per-core comparison with LAL and waveformrs is still to be made
- NRSur7dq4's float32 timings on an idle machine: they were measured while other jobs ran (`report/float32_nrsur7dq4.py --only-timings`; the NRSur7dq4 section)

IMRPHENOMXPNR AGAINST LAL 6.2.0

$\leq 5.1\times10^{-5}$

Largest relative error, $|h - h_{LAL}| / \max |h_{LAL}|$, over 193 random binaries (median 6.0×10^{-7}); largest mismatch 2.1×10^{-9}. With the parity fix, against LAL patched with it: $\leq 4.7\times10^{-5}$.

NRSUR7DQ4 AGAINST LAL 6.2.0

$\leq 2.8\times10^{-12}$

Over 200 random binaries, with LAL's length and epoch in 200 of them; largest mismatch 3.3×10^{-16}.

NRSUR7DQ4 DERIVATIVES

≤ 2.9×10^{-8}

Largest disagreement of any column of the Jacobian with finite differences, over four binaries, wherever the waveform is differentiable. For IMRPhenomXPNR the finite differences disagree by up to 2.3×10^{-1}, which is still being investigated (section 8).

ONE IMRPHENOMXPNR WAVEFORM

11.92 ms

Compiled, on 8033 frequencies, against 13.97 ms for LALSimulation and 3.01 ms for waveformrs. The first call compiles for 17.7 s.

BATCHES WITH `JAX.VMAP`

112 / s

IMRPhenomXPNR waveforms per second at best over batch sizes, against 4,667 / s for waveformrs on all threads and 74 / s for LALSimulation on one.

TESTS

516 / 518 pass

pytest, including the slow random-draw tests, in 28 minutes. They compare every part of the models with LALSimulation and the reference data of waveformrs.

COMPARISON	DRAWS	MAX REL. ERROR	MEDIAN REL. ERROR	MAX MISMATCH	MEDIAN MISMATCH
IMRPhenomXPNR without the parity fix, against LALSimulation 6.2.0	193/200	5.1×10^{-5}	6.0×10^{-7}	2.1×10^{-9}	7.8×10^{-12}
IMRPhenomXPNR with the parity fix, against 6.2.0 + lalsuite !2593	193/200	4.7×10^{-5}	5.3×10^{-7}	3.0×10^{-9}	5.3×10^{-12}
IMRPhenomXPNR without the fix, against waveformrs	193/200	5.1×10^{-5}	6.0×10^{-7}	2.1×10^{-9}	7.8×10^{-12}
NRSur7dq4, against LALSimulation 6.2.0	200/200	2.8×10^{-12}	2.6×10^{-13}	3.3×10^{-16}	0
NRSur7dq4, against waveformrs	200/200	2.8×10^{-12}	2.6×10^{-13}	3.3×10^{-16}	0

Random binaries drawn as in waveformrs's report (section 7). IMRPhenomXPNR is evaluated on 4,096 uniformly spaced frequencies per draw. "Draws" counts the draws compared. Mismatches use the aLIGO design sensitivity without maximisation.

In short:

- **Implementation.** 93 Python modules (27,688 lines, of which 3,094 are parameter-space fits generated from waveformrs's Rust by a translator) cover the two models, their checks and the conversions (sections 2–5).
- **Accuracy.** NRSur7dq4 agrees with LAL at rounding level. IMRPhenomXPNR's (2,2) mode agrees to ~1e-9. Its higher modes agree to ~1e-5, because LAL takes derivatives by finite differences whose rounding noise JAX's different, but valid, arithmetic changes (section 7).
- **Single precision.** NRSur7dq4 and IMRPhenomXPNR run in float32 when the binary's parameters are float32, and return the strain times 10^{21} by default. For NRSur7dq4, In float32, an inner product between two different waveforms changes by 2.4×10^{-6} of the product of their norms (median; at most 1.0×10^{-4}). Derivatives agree with float64 to 5.6×10^{-5} (section 4).
- **Apple GPUs.** NRSur7dq4 also runs in MLX on Apple GPUs, in float32, with its derivatives, LAL's grids and the conditioned waveforms. It is as accurate as JAX's float32 version: the overlap of two binaries changes by 2.3×10^{-6} (median). In batches it evaluates 41,582 waveforms per second, 27.5× JAX's best on this machine's CPU (section 4).
- **IMRPhenomXPNR on Apple GPUs.** The lean IMRPhenomXPNR runs in MLX, in float32, with its derivatives, generated from the JAX model's own jaxprs. Its mismatch with float64 is as small as JAX's float32 one (5.8×10^{-9} median). In batches it evaluates 3,054 waveforms per second, 28.1× JAX's best on this machine's CPU (section 3).
- **Derivatives.** IMRPhenomXPNR is differentiated in forward mode, and its reverse-mode gradient is the transpose of that forward-mode Jacobian. NRSur7dq4 is differentiated directly in either mode. NRSur7dq4's derivatives agree with finite differences to ~1e-8. For IMRPhenomXPNR, finite differences of the full waveform disagree with the derivatives by up to 2.3×10^{-1}, most in the in-plane spins; the transform tests, which use the (2,2) mode, agree to ~1e-7. This is still being investigated (section 8).
- **Cost.** A compiled IMRPhenomXPNR waveform costs a little less than LALSimulation's, and about twice as much as waveformrs's. `jax.vmap` over many binaries is how JAX code would use it. Compilation takes 15–20 s for the model, 20–25 s under `jax.vmap` and under a minute for its gradient (section 9).

Two bugs found while preparing this report, both fixed. First, the conversions and the IMRPhenomXPNR speed-ups had been developed on separate branches. After the merge, `jax.grad` of `simulate_td_function` failed with "Linearization failed to produce known values". The conversions called the model through a reverse-mode wrapper, while the speed-ups had made the model's loops forward-mode only. The conversions now use the model's forward-mode Jacobians, with respect to the binary and to the starting frequency, which itself depends on the binary (`deeffce`). Second, when the first use of NRSur7dq4's data was the pure function under `jax.jit`, the cached data were built inside the trace and leaked tracers into later calls. The tests had always read the data through a plain call first; the gradient measurements of section 8 did not (`0a93b55`, with a test).

2 Models as pure JAX functions

The package has the interface of waveformrs's Python bindings, with the same function names, keyword arguments, options, result classes and exceptions. Underneath, every model is a pure function of a `Binary` : a `NamedTuple` of float64 arrays (or float32 arrays), and so a JAX pytree.

```
import jax, jax.numpy as jnp, numpy as np
import waveformjax as wf

frequencies = np.arange(20.0, 1024.0, 0.125)
hp, hc = wf.polarizations("IMRPhenomXPNR", frequencies, mass_1=36.0, mass_2=29.0,
                          spin_1=(0.4, 0.2, 0.3), spin_2=(-0.2, 0.3, -0.1), f_ref=20.0)

# the pure function, for jit, vmap and grad
model = wf.polarizations_function("IMRPhenomXPNR")          # f(binary, frequencies)
binary = wf.Binary(mass_1=36.0, mass_2=29.0, spin_1=jnp.array([0.4, 0.2, 0.3]),
                   spin_2=jnp.array([-0.2, 0.3, -0.1]), f_ref=20.0)
loss = lambda b: jnp.sum(jnp.abs(model(b, frequencies)[0]) ** 2)
jax.grad(loss)(binary)                                      # a Binary of derivatives
jax.vmap(model, in_axes=(0, None))(binaries, frequencies)    # many binaries at once

# NRSur7dq4 gives the strain times 1e21 (scaled_strain=False: the strain itself)
h = wf.td_polarizations("NRSur7dq4", 1 / 4096, 20.0, mass_1=40.0, mass_2=20.0)
surrogate = wf.td_polarizations_function("NRSur7dq4")       # f(binary, times)
single = wf.Binary(mass_1=jnp.float32(40.0), mass_2=jnp.float32(20.0), spin_1=jnp.zeros(3, jnp.float32),
                   spin_2=jnp.zeros(3, jnp.float32), f_ref=20.0)
hp, hc = surrogate(single, h.times)                         # in float32
h = wf.simulate_td("IMRPhenomXPNR", 1 / 4096, 20.0, mass_1=36.0, mass_2=29.0)
```

A call goes through four stages:

1. **Checks on the host**
   The public functions convert the arguments and run LALSimulation's checks on concrete values (`MODEL.validate`), outside any JAX transformation. Invalid masses, spins or frequencies, or a precession version LAL would replace while `version_fallback=False` , raise the exceptions of waveformrs: `InvalidParameterError` , `PrecessionVersionUnavailableError` and so on.
2. **A compiled function per configuration**
   The model method is compiled with `jax.jit` with the options fixed. `WaveformOptions` is a frozen dataclass, so it can be a static argument. JAX caches the compiled function per options and array shape, so a sampler that keeps its frequency grid compiles once.
3. **Traced arithmetic**
   Inside, nothing branches in Python on the binary. Each of LAL's `if` s becomes a `jnp.where` between branches that are both evaluated, with safe arguments, so that the branch not taken cannot put NaN into the value or the gradient. Loops of fixed length are `lax.scan` ; loops whose length depends on the binary are `lax.while_loop` . The precession prescriptions are branches of a switch.
4. **Failures as NaN**
   Failures that LAL can only detect while computing, such as a SpinTaylor integration that cannot reach the reference frequency, make the output NaN. The batch functions report those binaries as `ModelError` in `errors` and set their rows to zero.

Importing the package enables 64-bit floats (`jax_enable_x64`), which IMRPhenomXPNR's comparisons with LAL need; both models also run in float32 (section 3, section 4). The thousands of fitted coefficients of IMRPhenomX are not copied by hand. `scripts/rust_to_python.py` parses the straight-line Rust of waveformrs (its `let` bindings, arithmetic, `fma` calls and `f64` methods) and emits the same expressions on JAX arrays. `scripts/generate_fits.py` then writes them into `models/xpnr/_generated/` . The control flow is written by hand, file by file after LAL's, and each module's docstring names the LAL function it follows.

Each model package defines a `MODEL` with a fixed contract. A frequency-domain model has `validate` , `polarizations` , `modes` and `choose_fd_modes` . A time-domain model has `validate` , `td_polarizations` , `td_modes` and `polarizations_at` , the last of which evaluates the waveform at any given times. The public functions, the batches and the conversions are written once, against that contract.

3 IMRPhenomXPNR

IMRPhenomXPNR is IMRPhenomXPHM with the phenomenological PNR precession angles, a co-precessing (2,2) mode tuned to numerical relativity, an antisymmetric (2,|2|) contribution and the 122019 release of the higher modes. In LALSimulation it draws on many of the 49 IMRPhenomX source files, together with the SpinTaylor code and GSL. Here it is about 14,800 lines of Python, of which 4,400 are generated fits.

Four prescriptions in one compiled function

LALSimulation chooses the Euler angles by the binary. In-plane spins below 1e-7 give plain IMRPhenomXPHM with the NNLO angles, without the PNR machinery. Below 1e-6, the PNR machinery runs with the NNLO angles (version 102). Otherwise the SpinTaylor angles are used (version 330). Where LAL's SpinTaylor integration fails, it falls back to the MSA angles (223), or to NNLO if the MSA system cannot be set up. Here the decision is a traced index, and the prescriptions are three branches of a switch; versions 102 and 223 of the PNR machinery share one branch, chosen by a traced flag. Under `jax.vmap`, `lax.switch` with a batched index would compute every branch for every binary. The model's own `switch`, `cond` and `while_loop` (`_transforms.py`) instead have batching rules: a branch is computed for the whole batch only if some binary in the batch takes it, and a loop runs to the longest binary in the batch while selecting only its small carry at each iteration.

The SpinTaylor angles

The SpinTaylor angles are the hardest part to make traceable. LAL integrates the orbit-averaged SpinTaylorT4 equations from `f_ref` backwards to below `f_min` and forwards to the end of the inspiral. It uses GSL's adaptive RKF45 at tolerances of 1e-12, samples the solution every $\Delta t = 5 / f_{cut}$, repeats the end with ten times finer sampling, and passes all the samples, millions for light binaries, to natural cubic splines of α and cos β. Where each run ends, and so how many samples there are, depends on the exact sequence of steps. The JAX version therefore splits the work:

- **The forward and fine runs** repeat LAL's adaptive steps one for one in a `while_loop` (`spintaylor/replica.py`): GSL's step control, the retries after a failed derivative, the Hermite-like interpolant and the stopping test. The run then ends where LAL's ends, with the same samples. Its derivatives are those of the recorded steps with the step sizes held fixed.
- **The backward run** is long and only ends at a frequency, so it is solved with Dormand–Prince 8(5,3) and its seventh-order dense output. The steps are equal increments of $u = 1 / (M\omega)$, so each step covers about the same precession angle, and a few hundred steps resolve a whole inspiral to ~1e-12. The step size is a smooth function of the state, so the solution is differentiable. It differs from LAL's samples by LAL's own integration error, ~1e-9.
- **The splines** have as many knots as LAL has samples. Up to 8192 knots they are built as GSL builds them. Beyond that, a natural cubic spline at x depends on knots k intervals away only through factors $\sim(2 - \sqrt{3})^k$. It is then evaluated from the 34 knots around x, which gives the same values to ~1e-15 at a cost per point independent of the number of samples.

Reproducing LAL's discrete choices

Much of LAL's code makes decisions on values that are themselves results of iterations or of NaN arithmetic. Each has to be reproduced exactly, or the waveform changes discontinuously:

- **MSA angles.** GSL's Jacobi elliptic function and elliptic integral are ported with a fixed number of iterations, and the iterations GSL would not do are masked out. Unlike LAL, which stores the roots of the spin cubic in a shared struct and races on them under OpenMP, every value here is local, so the angles agree with single-threaded LAL.
- **Higher modes.** The order of the intermediate amplitude polynomial is lowered when it would cross zero. `CrossZeroP5` takes the roots from GSL's `gsl_poly_complex_solve`, and LAL's indexing of the roots depends on their order, so GSL's balanced companion matrix and Francis QR iteration are ported step for step in `while_loop`s that end where GSL's end.
- **PNR β connection.** LAL picks the connection frequencies of β from the turning and inflection points of a cubic. The computation often produces NaN (complex roots, square roots of negative numbers), which LAL's comparisons then treat as false. It is evaluated twice. The first pass uses LAL's arithmetic, NaN included and under `stop_gradient`, to make the decisions. The second uses finite stand-ins that follow those decisions, to give the values and their gradients. The inflection points are roots from the trigonometric formula, close to an `acos` of 1, so there LAL's fused multiply-adds are reproduced exactly, giving ~1e-12 rather than ~1e-8.
- **Antisymmetric amplitude.** LAL smooths the amplitude ratio with an 81-point moving average computed in place, so each window reads averages it has already written. That is an order-40 linear recurrence. It is evaluated in blocks of 40 values with a triangular solve per block, inside a `lax.scan`.

Derivatives in forward mode

Loops whose number of iterations is traced, such as the SpinTaylor runs and GSL's QR iteration, can be differentiated in forward mode but not in reverse mode. Reverse mode through the model's many scalar set-up computations also compiles slowly. So `_transforms.differentiable` makes the model a `jax.custom_jvp` function. Its rule computes the Jacobian with respect to the parameters that have non-zero tangents, one `jax.jvp` per parameter, and contracts it with the tangents. The contraction is linear, so `jax.grad`, which transposes it, works as well as `jax.jvp` and `jax.jacfwd`. The Jacobian is itself a `custom_vmap` function, so under `jax.vmap` of a gradient the binaries are batched inside the differentiation. Where LAL takes derivatives by finite differences, with steps of 1e-9 and 1e-7 in the intermediate region of the higher modes, the values follow LAL. Their tangents, however, are those of the exact derivatives. The tangents of the finite differences themselves would amplify rounding by up to 1e14.

The parity fix

By default the model includes the fix of lalsuite !2593 from LALSuite's development branch, as waveformrs does. The fix restores the mirror symmetry of the antisymmetric modes: flipping the in-plane spins and taking $\iota \to \pi - \iota$ gives $(h_+, -h_\times)$. `pnr_parity_fix=False`, or `WaveformOptions.lalsimulation_defaults()`, gives the released LALSimulation 6.2.0. Both are compared with LAL in section 7, the fixed model with a LALSimulation 6.2.0 patched with the fix (UNCLEAN: lalsimulation-v6.2.0 + lalsuite!2593 (git diff 64d2004def^1 64d2004def -- lalsimulation/lib)).

Making it compile and run faster

The first complete version took about 32 ms per waveform and 2.3 s for `jax.vmap` over 64 binaries; compiling its reverse-mode gradient took 714 s. A second round of work brought these to about 18 ms, 0.74 s and 132 s (measured during development on this machine). It moved the derivatives to forward mode and added the batch-aware branches and loops. It merged versions 102 and 223 into one branch, computed only the PNR β of the binary's region, chose the intermediate polynomials of all higher modes together, and solved the SpinTaylor equations with the 3-vectors as arrays. The current numbers are in section 9.

A third round cut the compilation further. Only the SpinTaylor prescription is compiled by default (`version_fallback=False`). Compile time grows with the number of instructions XLA optimises, and XLA already merges identical computations, so the round looked for code compiled more than once with different inputs. The SpinTaylor splines are now evaluated at all the points the set-up reads in one call, and at the frequencies of the antisymmetric mode together with the PNR grid. The six derivative evaluations of an RKF45 step, the double QR sweep of GSL's root finder and the coefficients of the four higher modes (under `jax.vmap`, with the mode a traced value) each compile once. The intermediate polynomials of the higher modes come from a small linear solve rather than LAL's closed forms. The co-precessing modes and the factors of the twist pass optimization barriers: XLA had copied their computation into both polarizations and into every column of the Jacobian, which made its fusion pass quadratic for the gradient. Along the way `CrossZeroP5` was fixed (section 10). Measured one at a time during development, tracing, lowering and compiling IMRPhenomXPNR (all modes, 64 frequencies) went from 25 s to 15 s, under `jax.vmap` from 35 s to 20 s, and its gradient from 115 s to 55 s. The gradient compiles a forward-mode Jacobian, about 3.4 times the instructions of the model, so it stays well above the model.

The same round made the compiled waveform about a quarter faster. XLA's CPU runtime runs a loop body on the calling thread only if the body has at most eight kernels or touches no buffer larger than 512 bytes. Otherwise it hands every iteration's kernels to a thread pool, at a cost of microseconds per iteration. The loop bodies of the SpinTaylor replica and of the DOP853 integration now stay within that limit: each step runs in a one-iteration loop that carries the state itself (`_math.isolated_step`). The cross products of the SpinTaylor equations use gathers rather than concatenations, which XLA makes a kernel each. The moving average of the antisymmetric amplitude, a triangular solve per block of 40 frequencies, solves one system for all blocks on a uniform frequency grid. Per step, the replica went from 8.5 to 6 µs and the DOP853 integration from 15 to 8.5 µs.

The lean model

The option `lean=True` (`models/xpnr/lean.py`) is the extended model of `report/extended`, with the direct SpinTaylor angles and the parity fix, stripped of the layers that only reproduce LAL's numerical procedure. It is the version meant for further work on speed and for ports to other frameworks. LAL's physics is kept: the fits, the ansatzes, the PNR construction of the angles and the discrete choices of the higher-mode amplitudes. What goes:

- **The SpinTaylor replica.** The precession equations are solved once, in one DOP853 loop with dense output (`spintaylor/lean.py`): backward steps of equal increments of $u = 1/(M\omega)$ to below f_{min}, then forward steps from f_{ref} (at most a tenth of u each, which keeps the steps short near $v = 1$ for large aligned spins) until LAL's stopping conditions are met. There is no RKF45 replay, no sampling lattice, no finer re-run of the end of the inspiral and no second integration from the start of the grid of the PNR interpolants: the one integration starts low enough for that grid. The end of the inspiral is the extended model's: where the first stopping condition is met on the dense output, less half a step of LAL's fine sampling, found by bisection and refined by a Newton step that carries the derivatives.
- **Arithmetic emulation.** Masses stay in solar masses, the mixing coefficients of the (3,2) mode are not rounded to single precision, nothing keeps XLA from fusing multiply-adds, and the connection frequencies of the PNR β follow the finite evaluation alone, without replaying LAL's NaN arithmetic. The quintic of the higher-mode amplitudes is tested for a zero crossing by its signs at 257 points instead of GSL's companion-matrix QR iteration.
- **Prescriptions and switches.** Only the SpinTaylor angles are compiled; the higher modes take exact derivatives where LAL takes finite differences (as the extended model does), evaluated at one point.

Over 1,400 random binaries drawn as in `report/extended`, the lean model agrees with the extended model to a largest mismatch of 1.0×10^{-9} (median at rounding), with the same binaries finite; at special binaries (aligned, single, vanishing and nearly aligned spins, a reference frequency beyond the PN energy test, $q = 18$, $\chi = 0.98$) the mismatches are at most $\sim10^{-12}$, and its Jacobian agrees with central finite differences to $\sim10^{-6}$ except across the zero of the in-plane spins, where the extended model is discontinuous by design. Against LAL, over 300 binaries for which LAL keeps the SpinTaylor angles, the weighted mismatch has median 1.9×10^{-8} and maximum 7.4×10^{-4}, as for the extended model (`report/extended`). In float32 it agrees with float64 to mismatches of 10^{-9}–3×10^{-7}. Its derivatives are finite in float32 (over 300 random binaries up to $q = 30$, each mode and the polarizations; before two fixes described below, the derivative of the (2,1) mode was NaN for half of them, mostly above $q = 10$, and that of every mode for 2%). Measured one at a time (`report/lean_xpnr.py`: 64 frequencies, all modes, tracing + lowering + compilation), the lean model compiles in 10.9 s against 14.4 s for the default model, under `jax.vmap` in 14.6 s against 19.6 s, and its gradient in 48.7 s against 56.7 s; at the benchmark binary it runs in 9.98 ms against 10.85 ms (the extended model: 17.29 ms).

Single precision

IMRPhenomXPNR is evaluated in the precision of the binary's parameters, as NRSur7dq4 is, and returns the strain times 10^{21} unless `scaled_strain=False`. The comparisons with LAL use `scaled_strain=False`. A float32 binary is evaluated without promoting any float32 value to float64. Python constants in LAL's expressions remain weakly typed float64 scalars, which take float32 when they meet the data; `tests/test_xpnr_float32.py` checks the waveform, its forward derivative and its gradient. The changes that made float32 work change the float64 waveform by rounding only:

- **No SI round trips.** In float64 the masses still pass through kilograms as in LAL. In float32 they stay in solar masses (`_math.mass_unit`), because XLA folds the square of the solar mass in kilograms into the derivatives, as it did for NRSur7dq4. The amplitude takes the distance in Mpc. Two expressions of LAL that XLA rewrites into powers beyond float32's range are written another way: the velocity of the chirp-time bound, where XLA turned a division by c into a multiplication by c^8, and the arctan window of the PNR β, whose 500th powers overflow. The 500th power in that window, below 1.5 for β in [0, 0.01], exceeds float32's range where the ringdown β of the fits is below −0.3 rad (large mass ratios with in-plane spins): its value is finite but its derivative was NaN, so there it is computed as π/2 − arctan of the inverse power, the same function to rounding. The pseudo-PN coefficients of the higher-mode inspiral amplitudes divide

by a product of powers of the collocation frequencies, ~10^{-19} for the (2,1) mode at Mf ~ 0.01, whose square in the derivative of the quotient falls below float32's range (NaN derivatives of the (2,1) mode, mostly above q = 10); the coefficients do not depend on the scale of the frequencies, and are computed in units of the end of the inspiral.

- **Constants of the precision.** Machine epsilon, the smallest normal number and the floors of safe square roots (10^{-300} in float64) follow the precision. The tolerances of the RKF45 replica (10^{-12}, below float32's rounding) become 10^{-5}, and the higher modes take exact derivatives instead of LAL's finite differences with a step of 10^{-9}.
- **Types from the binary.** Default float and integer types, numpy coefficient tables and the counts of loops follow the precision (`_math.real` , `_math.integer`), since an int64 count or a float64 table promotes float32 data to float64.

The extended model in float32 agrees with its float64 waveform to a mismatch of ~10^{-9}–10^{-7} for the binaries tried, and its gradients agree to ~10^{-4}. The default model replays LAL's adaptive RKF45 steps and samples them on LAL's time lattice. In float32 its waveform differs from float64 by mismatches of 10^{-5}–10^{-2}, so float32 is meant for the extended model. Running JAX's float64 switch off inside a trace (`jax.enable_x64(False)`) would have avoided most of the type changes, but JAX caches the conversions of numpy constants and the executables of eager operations without regard to that switch, so a float32 evaluation corrupted later float64 ones in the same process.

The lean model on Apple GPUs with MLX

`waveformjax.models.xpnr.mlx` runs the lean model in MLX on the GPU of Apple silicon, in float32: `polarizations(binary, frequencies)` for one binary or a batch (shared or per-binary frequencies), differentiable with `mx.grad` , and `polarizations_and_jacobian` . It is not a second implementation of the model. The lean model is cut into stages that are functions of one binary's scalars (the set-up: four stages of 7–41 thousand scalar operations) or of its scalars and one point (the PNR angles at a grid point, the angles, modes and twist at a frequency). Each stage is traced to a jaxpr in float32 and lowered (`mlx/_program.py`) to straight-line code on scalars: arrays of static shape become arrays of scalars, loops of static length are unrolled, branches are computed both ways and selected, the small linear solves (LAPACK's `getrf` with partial pivoting) and sorts are written out, complex numbers become pairs, and what does not depend on the inputs is computed once. The code runs as a Metal kernel over the batch and the points, its derivatives from the JVP that JAX writes, one thread per binary, point and direction. Products added once are fused into multiply-adds, as XLA's CPU backend fuses them: without this, the float32 fits of the PNR β and the collocation solves of the higher modes came out 20–100 times further from float64 than JAX's float32. The rest is written for MLX: the SpinTaylor integration (a Metal kernel, one SIMD group per binary with a lane per component of the state, calling the translated derivatives), the searches in its dense output, the natural splines of the PNR interpolants (the inverse of the tridiagonal matrix as a filter of the second differences), Boole's integral of γ and the moving average of the antisymmetric mode.

The derivatives are forward-mode Jacobians with respect to the twelve parameters, contracted with the cotangents in a custom VJP, as the JAX model contracts its own. One choice differs from JAX's float32: the Newton step at the end of the inspiral takes the exact derivative of the stopping conditions in time, where JAX takes the secant of the bisection's bracket, which float32 does not resolve. JAX's float32 derivative of the end of the inspiral is off by up to 90% for the binaries tried; the MLX one agrees with float64 to ~10^{-6}.

Over 200 random binaries of `report/extended` (1024 frequencies each), the weighted mismatch of the MLX waveform against JAX's float64 lean waveform has median 5.8×10^{-9}, 90th percentile 5.0×10^{-7} and maximum 1.1×10^{-2}; JAX's float32 lean waveform: 9.4×10^{-9}, 3.4×10^{-7} and 1.1×10^{-2}. 14 binaries exceed 10^{-6} in MLX and 11 in JAX's float32; the largest is the same binary of 411 $M_\odot$ for both (JAX float32: 1.1×10^{-2}). Where they differ, the cause is float32 itself, not the port: LAL's connection of the PNR α takes finite differences of the SpinTaylor α over 0.0005 in M f, which float32 resolves to ~10^{-2}, so its rounding moves the connected α by up to a few hundredths of a radian, more in one evaluation or the other. Both are finite for 200 binaries; 0 are finite in float64 only and 0 in MLX only (1 not in JAX's float32). The gradient of the log-likelihood of `tests/test_xpnr_float32.py` is within that test's bound (10^{-2} |g| + 10^{-3} max |g|) for 26 of 30 binaries, at 5.8×10^{-2} of the bound (median; largest 7.4×10^{0}), against 22 of 30 for JAX's float32 (2.8×10^{-1}, largest 5.9×10^{1}). Data: `report/data/mlx_xpnr.json` (`report/mlx_xpnr.py`).

BINARIES PER CALL	MLX, PER CALL	WAVEFORM, PER BINARY			GRADIENT, PER BINARY		
		MLX	JAX FLOAT32	JAX FLOAT64	MLX	JAX FLOAT32	JAX FLOAT64
1	21.65 ms	21.65 ms	9.20 ms	10.30 ms	120 ms	110 ms	118 ms
16	68.46 ms	4.28 ms	22.76 ms	26.11 ms	41.47 ms	180 ms	289 ms
64	95.41 ms	1.49 ms	40.00 ms	56.94 ms	23.53 ms	–	–
256	152 ms	594 µs	–	–	23.17 ms	–	–
1024	335 ms	327 µs	–	–	–	–	–

The lean IMRPhenomXPNR on 8033 frequencies (20–1024 Hz), all modes: MLX on the Apple M2 Ultra GPU, JAX on the CPU under `jax.jit` (one binary) and `jax.jit(jax.vmap(...))` (random binaries of 20–120 $M_\odot$ from 20 Hz). The gradient is that of the power of the polarizations. Median times; the larger JAX batches were not timed.

One waveform takes 21.65 ms in MLX against 10.30 ms in JAX (float64) on the CPU: a single binary is dominated by the sequential integration (one SIMD group runs its steps one after the other) and by launching the kernels. In batches MLX evaluates 3,054 waveforms per second (at 1024 binaries per call), 28.1× JAX's best on the CPU (109 per second), and 43 gradients per second (at 256), 4.8× JAX's best (9.1). The gradient is a Jacobian along twelve directions, each thread recomputing the values: at large batches its time goes to the PNR angles on the grid of the interpolants and to the angles and modes at the frequencies. In a new process, importing takes 535 ms, the first waveform 9.6 s (tracing the stages, lowering them and compiling their kernels) and the first gradient 4.2 s (the JVP kernels); JAX compiles the lean model in 10.9 s and its gradient in 48.7 s.

4 NRSur7dq4

NRSur7dq4 interpolates numerical-relativity waveforms of precessing binaries. It was trained on mass ratios up to 4 and spin magnitudes up to 0.8, and LAL allows it to extrapolate beyond those. It integrates an ODE for the coprecessing frame, the orbital phase and the spins, with time derivatives given by fits at 233 nodes. It then evaluates the coorbital modes from fits at their empirical-interpolation nodes and rotates them to the inertial frame. The JAX version follows `LALSimIMRPrecessingNRSur.c` and waveformrs's translation of it.

- **Data.** The 17 MB HDF5 file is read once with h5py. Every dynamics fit becomes a dense row over the union of the monomials the fits use, so the nine fits of a node are one matrix–vector product. The fits of the waveform nodes are stored by the order of their mass-ratio monomial, so the mass ratio, which is the same at every node, is folded into the coefficients once per binary.
- **Dynamics.** LAL steps from the reference frequency to the nearest node, takes three RK4 steps (forwards or backwards, depending on that node), and covers the remaining nodes with fourth-order Adams–Bashforth steps in both directions. Since the starting node depends on the binary, the integration is one `lax.scan` of fixed length. It walks backwards from the start and then forwards, evaluating the fits of one node per step as LAL does. The Adams–Bashforth weights depend only on the node and are tabulated once.
- **Splines.** GSL's natural cubic splines solve a tridiagonal system whose matrix depends only on the knots. As in waveformrs, it is factorised once when the data are read, so fitting a spline needs only the two substitution sweeps.
- **Frame rotation.** LAL's Wigner D-matrices use negative powers of the quaternion's components, with special cases where they vanish, as they do for spins along the orbital angular momentum. Here the negative powers are cancelled analytically, which makes each element a polynomial in the quaternion: the same values, no special cases, and finite gradients in the non-precessing limit.
- **Output grid.** The number of samples depends on when the orbital frequency reaches $\pi\, f_{\mathrm{min}}$, so it is found on the host. A compiled pass evaluates the frequency fits, and numpy repeats LAL's search operation by operation. The waveform is then computed on a power-of-two buffer of at least 256 samples, which waveforms of similar length share, and trimmed to the binary's length.

`polarizations_at(binary, times)` evaluates the waveform at any times relative to the peak. It is pure and differentiable with respect to every parameter, including `f_ref`, which moves the reference time and so the frame. It is what `jax.vmap`, `jax.grad` and the conversions use.

Single precision and the scale of the output

NRSur7dq4 is evaluated in the precision of the binary's parameters. A `Binary` of float32 arrays is evaluated in float32 throughout, with a float32 copy of the data cast from the float64 arrays (the spline factorisations included). A test checks that no operation of the traced waveform, or of its gradient, is in float64. The public functions take Python numbers, so they stay in float64. Two changes make float32 usable:

- **The output is the strain times 10^{21}.** This applies to every NRSur7dq4 function, the public functions and the conversions included, unless `scaled_strain=False`. The strain itself fits in float32, but its square (~10^{-42}) and a detector's power spectral density (~10^{-47}) do not. The comparisons with LAL in this report use `scaled_strain=False`.
- **Nothing passes through SI units.** LAL converts the masses to kilograms and back (`m * MSUN_SI / MSUN_SI`) and works with the distance in metres. In float32, XLA folded the square of the solar mass in kilograms (~4×10^{60}, beyond float32's range) into the derivative of the mass ratio, so the compiled gradient was NaN. The derivative with respect to a distance of ~10^{25} m underflowed to zero. The masses now stay in solar masses, and the amplitude is one constant times the total mass over the distance in Mpc. In float64 this changes the waveform by rounding only.

What float32 costs in accuracy was measured on 300 pairs of binaries, drawn as for the accuracy comparison (`report/float32_nrsur7dq4.py`). 150 pairs are two independent binaries, whose overlaps are near zero. The other 150 are a binary and a small perturbation of its masses, spins and angles, with a median overlap of 0.95. Both binaries of a pair are evaluated on one time grid, in float64 and in float32, projected on a detector and tapered at the start. Inner products use the aLIGO design sensitivity from 20 Hz, without maximisation, and are computed in float64 unless stated otherwise.

CHANGE FROM FLOAT64 TO FLOAT32	MEDIAN	90TH PERCENTILE	LARGEST
Inner product of the two waveforms, change over the product of their norms	2.4×10^{-6}	2.0×10^{-5}	1.0×10^{-4}
Normalised overlap, independent binaries	9.9×10^{-7}	8.0×10^{-6}	3.9×10^{-5}
Normalised overlap, nearby binaries	4.1×10^{-6}	3.0×10^{-5}	1.0×10^{-4}
Normalised overlap, with the inner products in float32 too	2.1×10^{-6}	1.9×10^{-5}	1.0×10^{-4}
Normalised overlap, with the dynamics in float64	2.8×10^{-7}	1.7×10^{-6}	4.6×10^{-6}
One waveform, noise-weighted relative error $\|\delta h\| / \|h\|$	3.8×10^{-5}	8.3×10^{-5}	2.5×10^{-4}
One waveform, largest relative error max $\|\delta h\|$ / max $\|h\|$	5.1×10^{-5}	1.3×10^{-4}	7.5×10^{-4}
One waveform, relative error of $\langle h, h\rangle$	6.4×10^{-7}	3.1×10^{-6}	2.9×10^{-5}
Derivatives of a log-likelihood, relative error (40 nearby pairs)	5.6×10^{-5}	6.0×10^{-4}	4.5×10^{-1}
Derivatives of a log-likelihood, error over the typical size of the component	6.5×10^{-7}	3.1×10^{-5}	2.0×10^{-3}

NRSur7dq4 in float32 against float64. The overlaps and inner products are between the two binaries of a pair. "With the inner products in float32 too" also does the whitened Fourier transforms and sums in float32. "With the dynamics in float64" integrates the ODE in float64 and computes the rest in float32. The log-likelihood is $\langle d, h\rangle - \langle h, h\rangle/2$, with noise-free data d from the other binary; its derivatives exclude f_{ref}, which is zero for half the pairs. Data: `report/data/float32_nrsur7dq4.json`.

An inner product between two different waveforms changes by 2.4×10^{-6} of the product of their norms (median), and by at most 1.0×10^{-4}. Doing the inner products themselves in float32 adds at most 1.7×10^{-7} to the overlap. The error of a single waveform, 3.8×10^{-5} (median, noise-weighted), is mostly rounding noise from the final resampling onto the float32 time grid: with everything up to the polarizations on the surrogate's own times in float64, it is still 2.5×10^{-5}. That noise is almost uncorrelated with another waveform, so it barely moves their inner product. The ODE integrated in float32 is what does. With the dynamics in float64 and the rest in float32, the change of the overlap falls from 2.1×10^{-6} to 2.8×10^{-7} (median), and from 1.9×10^{-5} to 1.7×10^{-6} at the 90th percentile.

Between nearby binaries the log-likelihood changes by $4.2\times10^{-6}\ \rho^2$ (median; $2.8\times10^{-5}\ \rho^2$ at the 90th percentile), where ρ is the signal-to-noise ratio. That is 1.7×10^{-3} at $\rho = 20$ and 4.2×10^{-2} at $\rho = 100$ (2.8×10^{-1} at the 90th percentile). With noise in the data, a further $\langle n, \delta h\rangle$ of about $\rho\ \|\delta h\| / \|h\|$ is added. The float32 derivatives agree with the float64 ones to 5.6×10^{-5} (median relative error), or 6.0×10^{-4} at the 90th percentile. The largest relative errors are in components whose derivative almost vanishes; measured against the typical size of each component, no error exceeds 2.0×10^{-3}.

On this CPU, float32 is 1.1× as fast for one waveform (2.74 ms against 3.05 ms on 8192 samples). It is 1.7× as fast for a batch of 64 under `jax.vmap` (56.61 ms against 94.01 ms), and 1.4× for a gradient. The dynamics, a sequence of 230 small steps, take 48% of a float64 batch. Keeping them in float64 would make the inner products about ten times more accurate, at the cost of part of the speed-up on the CPU; the package does not do this. float32 matters most on accelerators, where float64 is much slower or, as on Apple GPUs, missing. The MLX version below runs there.

On Apple GPUs with MLX

`waveformjax.models.nrsur7dq4.mlx` is NRSur7dq4 in MLX, Apple's array library, which runs on the GPU of Apple silicon. It is an optional dependency (`pip install waveformjax[mlx]`). The module is separate from the JAX code, which it leaves unchanged, and reads the same data through the JAX loader. It provides:

- the pure functions `polarizations_at` and `modes_at`;
- LAL's waveforms on its grid: `td_polarizations`, `td_modes` and `batch_td_polarizations`;
- the conditioned waveforms: `simulate_td`, `simulate_fd` and their pure functions.

Everything is float32 and differentiable with `mx.grad`. The pure functions take one binary or a batch along a leading axis. MLX has no float64 on the GPU and no `scan`, `while_loop` or `cond`. Its `vmap` and forward-mode derivatives do not pass through custom kernels. So:

- **Splines as matrices.** The JAX version fits its splines with two substitution sweeps (a `lax.scan`). The knots are fixed, so each spline is a fixed linear map, computed in float64 when the data are read:
  - the spline of the dynamics states, evaluated at the waveform's times, is one 2000 × 230 matrix;
  - the second derivatives of the spline through the polarizations are the inverse of its tridiagonal system applied to the right-hand side.

  For a batch, each map is one matrix product over all the binaries.
- **The dynamics in one Metal kernel.** The 230 sequential RK4 and Adams–Bashforth steps run in a hand-written Metal kernel, with one SIMD group per binary and one state component per lane. A second kernel carries dual numbers through the same steps. It gives the Jacobian of the states with respect to the mass ratio, the spins and the reference time, and the kernel's VJP contracts that Jacobian.
- **The frame rotation in Metal kernels.** The Wigner rotation of every mode sample is a generated kernel. Its VJP is a kernel too: it differentiates with respect to the quaternion through the generators of the rotation, using $D(pq) = D(p)\,D(q)$.
- **Harmonics as Fourier series.** The spin-weighted harmonics are trigonometric series in the inclination, with coefficients computed once in float64. Evaluated directly, the powers of $\cos(\theta/2)$ and $\sin(\theta/2)$ in the gradient were fused by `mx.compile` into a kernel with more arguments than Metal allows.
- **LAL's grids on the host.** The lengths and epochs are computed in float64, as the JAX driver computes them. A batch is set up all at once: the frequency fits of every binary in one call, then LAL's search in numpy over the batch. This takes 2.96 ms for 1024 binaries, against ~0.15 ms per binary one by one. A binary that LAL refuses is set up again on its own, which raises LAL's error. For 1200 binaries the grids agree with the JAX driver's:

- from 20 Hz: the same length and epoch for 407 of the 407 binaries LAL accepts, and LAL's error for 193 of the 193 it refuses;
- from the start of the surrogate: the same length and epoch for 600 of the 600 binaries LAL accepts;
- the start times differ by at most 1.0×10^{-11} M, the rounding of the fits summed in another order.

The binaries of a batch are sorted by length and evaluated in chunks of a power-of-two size.

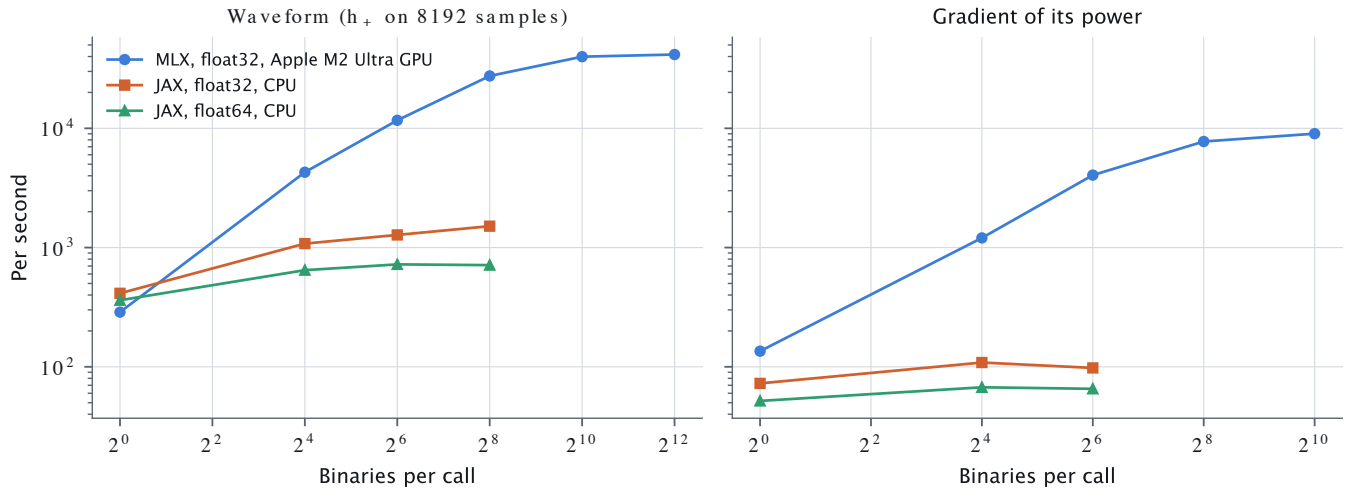
- **Conditioning.** LAL does not high-pass filter NRSur7dq4, so `SimInspiralTD` and `SimInspiralFD` need only its start taper, the padding and an FFT. The end of the taper depends on the samples, so it is found on the host from the float32 samples.

The accuracy was measured on the same 300 pairs as JAX's float32 above, in the same way (`report/mlx_nrsur7dq4.py`). The MLX waveform is evaluated by `polarizations_at` at the float32 times.

CHANGE FROM FLOAT64 JAX	MLX, FLOAT32			JAX, FLOAT32		
	MEDIAN	90TH PCT.	LARGEST	MEDIAN	90TH PCT.	LARGEST
Normalised overlap of the two binaries, change	2.3×10^{-6}	2.2×10^{-5}	1.2×10^{-4}	2.1×10^{-6}	1.9×10^{-5}	1.0×10^{-4}
Inner product over the product of the norms, change	2.4×10^{-6}	2.2×10^{-5}	1.2×10^{-4}	2.4×10^{-6}	2.0×10^{-5}	1.0×10^{-4}
One waveform, noise-weighted relative error ‖δh‖ / ‖h‖	5.2×10^{-5}	1.1×10^{-4}	2.2×10^{-4}	3.8×10^{-5}	8.3×10^{-5}	2.5×10^{-4}
One waveform, largest relative error max |δh| / max |h|	6.4×10^{-5}	1.6×10^{-4}	7.3×10^{-4}	5.1×10^{-5}	1.3×10^{-4}	7.5×10^{-4}
Derivatives of a log-likelihood, relative error (40 nearby pairs)	5.6×10^{-5}	4.8×10^{-4}	1.3×10^{-1}	5.6×10^{-5}	6.0×10^{-4}	4.5×10^{-1}
Derivatives of a log-likelihood, error over the typical size of the component	8.6×10^{-7}	5.6×10^{-5}	1.1×10^{-3}	6.5×10^{-7}	3.1×10^{-5}	2.0×10^{-3}

NRSur7dq4 in MLX on the GPU and in JAX on the CPU, both in float32, against JAX in float64, on the pairs of the table above. The inner products are computed in float64. JAX's derivatives are those of the table above. Data: `report/data/mlx_nrsur7dq4.json` .

The MLX waveform is as accurate as JAX's float32 one. The overlap of two binaries changes by 2.3×10^{-6} (median; 2.2×10^{-5} at the 90th percentile), against 2.1×10^{-6} (1.9×10^{-5}) in JAX. The derivatives agree with float64 to 5.6×10^{-5} (median relative error). Against the typical size of each component, no error exceeds 1.1×10^{-3}. The conditioned waveforms of 9 cases agree with the JAX conversions to 6.8×10^{-5} of the peak, with the same lengths, epochs and sampling in 9/9.



NRSur7dq4 waveforms (left) and gradients of their power (right) per second, against the number of binaries per call. MLX is on the Apple M2 Ultra GPU, compiled with `mx.compile` . JAX is on the CPU under `jax.jit(jax.vmap(...))` . Each waveform is h_+ on 8192 samples at 4096 Hz, for random binaries.

BINARIES PER CALL	MLX, PER CALL	WAVEFORM, PER BINARY			GRADIENT, PER BINARY		
		MLX	JAX FLOAT32	JAX FLOAT64	MLX	JAX FLOAT32	JAX FLOAT64
1	3.48 ms	3.48 ms	2.42 ms	2.77 ms	7.39 ms	13.77 ms	19.32 ms
16	3.73 ms	233 µs	927 µs	1.55 ms	830 µs	9.20 ms	14.86 ms
64	5.48 ms	86 µs	784 µs	1.38 ms	247 µs	10.22 ms	15.29 ms
256	9.31 ms	36 µs	662 µs	1.40 ms	129 µs	–	–
1024	25.67 ms	25 µs	–	–	111 µs	–	–
4096	98.50 ms	24 µs	–	–	–	–	–

Median times of the figure's calls. MLX 0.32.4 runs on the Apple M2 Ultra GPU (64 GB of unified memory); JAX runs on its CPU. The larger JAX batches were not timed.

One waveform takes 3.48 ms in MLX, against 2.42 ms in float32 JAX on the CPU. A single small waveform is dominated by the sequential dynamics and by launching kernels, so the GPU does not help. It pays off in batches:

- **Waveforms.** MLX evaluates 41,582 waveforms per second at 4096 binaries per call. JAX manages at best 1,511 in float32, so MLX is 27.5× faster.
- **Gradients.** One gradient takes 7.39 ms, against 13.77 ms in JAX. In batches MLX computes 9,018 gradients per second, 83.0× JAX's best in float32.
- **The public batch function.** It evaluated 1024 random binaries on LAL's grids, at 4096 Hz from the start of the surrogate (9,210 samples each on average), in 56.60 ms. That is 18,093 waveforms per second, against 548 for JAX's `batch_td_polarizations` (33.0×).
- **One binary.** `td_polarizations` takes 3.41 ms (JAX 3.11 ms), and `simulate_fd` takes 5.62 ms (JAX 3.69 ms).

In a new process, reading the data and preparing its float64 operators takes 1.9 s. The first waveform then takes 35.19 ms and the first gradient 13.73 ms. MLX builds its kernels as they are first used, which costs little here. JAX, by comparison, takes 2.4 s for its first waveform and 8.8 s to compile the gradient.

5 Time and frequency domains

`simulate_td` and `simulate_fd` reproduce LALSimulation 6.2.0's `SimInspiralTD` and `SimInspiralFD`. IMRPhenomXPNR can be given in the time domain and NRSur7dq4 in the frequency domain, and each model can also be conditioned in its own domain. They follow waveformrs's `src/conversions/` and `src/conditioning.rs`.

LAL's conditioning starts the model below the requested frequency by an amount set by bounds on the chirp time, merger time and ringdown time. It tapers the extra length, applies a Butterworth high-pass filter, shifts the waveform in time and crops it to a length and epoch derived from the same bounds. The work is split by what needs to be exact:

- **On the host, in Python floats.** The lengths, shifts, taper widths and GPS epochs are computed with LAL's arithmetic. This includes C's `pow`, `cbrt` and `round`, and the fused multiply-adds of clang's arm64 build, reproduced exactly through rational arithmetic. As a result, lengths and epochs are LAL's bit for bit.
- **In JAX, pure and differentiable.** The bounds are computed again where the waveform depends on them continuously: the starting frequency given to the model and the cutoff of the first high-pass filter. So do the filter coefficients, the filter itself (a `lax.scan` of LAL's recursion per second-order section), the tapers and the Fourier transforms. Arrays have fixed, power-of-two lengths, so one compilation serves many binaries.

IMRPhenomXPNR smooths its antisymmetric amplitude with a moving average over the array of frequencies, which LAL takes over its band. To keep one array length per transform length, the model is evaluated on the band, followed by frequencies just above it and then the band's last 82 frequencies again. The first L − 41 values are then averaged as LAL averages them, and the last 41 are taken from the end. `simulate_td_function` and `simulate_fd_function` return the conversion as a pure function. It keeps the lengths, shifts, tapers and epoch of a reference binary, while the starting frequencies and filter cutoffs follow the binary, so the gradient includes their dependence on the masses and spins.

6 How the package is tested

518 pytest tests (516 passed, 2 failed, 0 skipped in the run below) compare every layer of the models with LALSimulation 6.2.0, through its Python API or, for internal functions, through ctypes. They also reuse waveformrs's reference suites, which cover each regime switch and fallback of the models.

TEST FILE	TESTS	RESULT	TIME
`test_quick.py` The quick tier (`pytest`): IMRPhenomXPNR and the extended model with its derivative, NRSur7dq4	5	2 failed	43.9 s
`test_api.py` The interface: options, arguments, errors, spherical harmonics	9	9 passed	330 ms
`test_xpnr_xas.py` IMRPhenomXAS (2,2) mode against `SimIMRPhenomXASFrequencySequence`	11	11 passed	611 ms
`test_xpnr_hm.py` Higher modes (release 122019) against `SimIMRPhenomXHMFrequencySequenceOneMode`	25	25 passed	144.7 s
`test_xpnr_msa.py` MSA and NNLO angles and the precession set-up, GSL's elliptic functions	87	87 passed	32.5 s
`test_xpnr_pnr_helpers.py` PNR fits and angle helpers against LAL's internal functions (ctypes)	55	55 passed	23.8 s
`test_xpnr.py` IMRPhenomXPNR polarizations: waveformrs's reference suite, with and without the parity fix	98	98 passed	215.7 s
`test_xpnr_modes.py` IMRPhenomXPNR J-frame modes, `ChooseFDModes`, version fallbacks, checks	12	12 passed	140.5 s
`test_xpnr_random.py` IMRPhenomXPNR, 64 random binaries against LAL (slow)	64	64 passed	56.7 s
`test_transforms_xpnr.py` IMRPhenomXPNR under `jit`, `vmap`, `grad` and `jvp`	5	5 passed	194.5 s
`test_nrsur7dq4.py` NRSur7dq4 polarizations, modes, dynamics, errors, data file, batches	37	37 passed	29.4 s
`test_nrsur7dq4_random.py` NRSur7dq4, 64 random binaries against LAL (slow)	1	1 passed	7.5 s
`test_transforms_nrsur7dq4.py` NRSur7dq4 under `jit`, `vmap` and `grad`, removable singularities	11	11 passed	72.1 s
`test_nrsur7dq4_float32.py` NRSur7dq4 in float32 (no float64 operation, accuracy, `jit`, `vmap`, `grad`) and the scale of its output	14	14 passed	87.4 s
`test_nrsur7dq4_mlx.py` NRSur7dq4 in MLX against JAX: waveform, modes, batches, gradients, the Metal kernels, LAL's grids and the conditioned waveforms	24	24 passed	44.2 s
`test_xpnr_mlx.py` The lean IMRPhenomXPNR in MLX against JAX: the stages, the generated kernels, the SpinTaylor kernel, waveform, Jacobian and gradients	23	23 passed	219.3 s
`test_conversions.py` `simulate_td` and `simulate_fd`: lengths, epochs, filters, tapers against LAL	31	31 passed	38.5 s
`test_transforms_conversions.py` The conversions as pure functions under `jit`, `vmap` and `grad`	6	6 passed	89.3 s

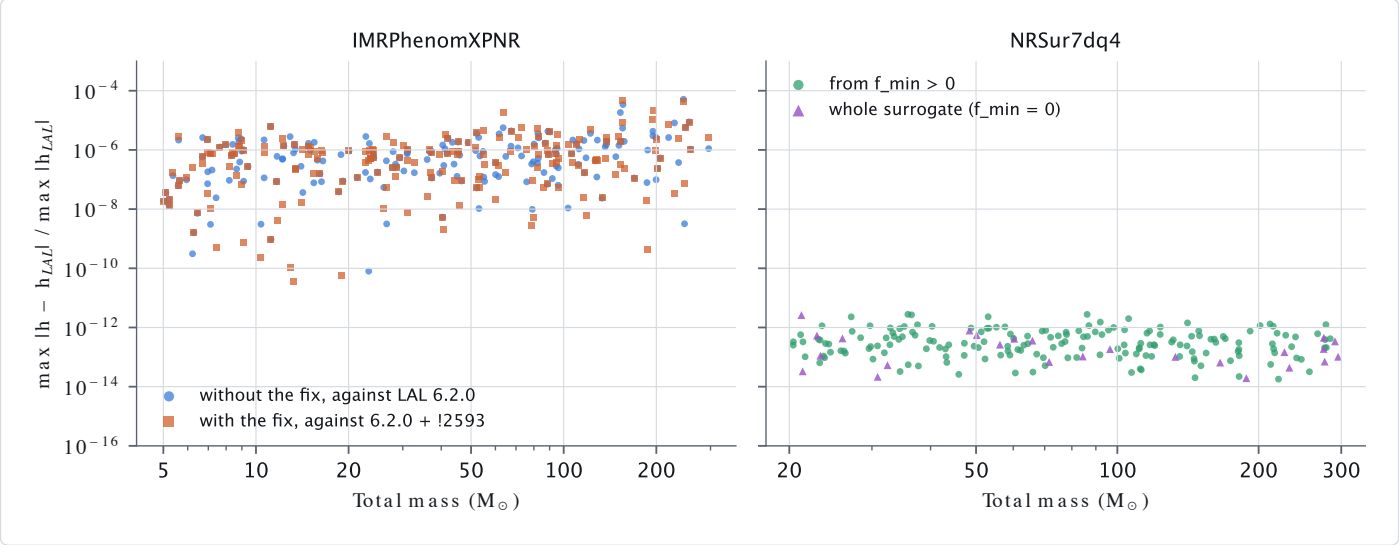
The full run with `WAVEFORMJAX_SLOW=1` (the random-draw tests) on the branch `xpnr-spintaylor-only`, with `OMP_NUM_THREADS=1` for LALSimulation. The MLX tests, which need MLX, come from their own run in a Python with MLX. Data: `report/data/pytest.xml` and `report/data/pytest_mlx.xml`.

The tests are layered as the model was built: the (2,2) mode, then the higher modes, the angles, the PNR helpers, the polarizations and the modes. Each layer is compared at a tolerance set by what LAL itself can resolve there. The MSA angles, for example, are ill-conditioned for nearly aligned spins, where LAL's own output changes by ~5e-8 for a change of the inputs in the last bit. Every model also has a transform test. It checks that `jax.jit` and `jax.vmap` give the plain call, that gradients agree with central finite differences for every parameter, and that they are finite at removable singularities such as aligned, vanishing or single spins.

The two failures are in the quick tier: the derivative of the extended model's (2,2) mode along an in-plane spin is NaN at a binary with aligned spins and at a non-spinning one, although the waveform itself is finite. The cause is not isolated yet; the extended model's own transform tests, at other binaries, pass.

7 Accuracy

The random draws are those of waveformrs's report, so the two reports can be compared. For IMRPhenomXPNR: total masses log-uniform in 5–300 $M_\odot$, mass ratios uniform in 1/20–1, isotropic spins of magnitude up to 0.99, $f_{min} \in \{10, 15, 20\}$ Hz, $f_{ref} = f_{min}$ or up to 3 f_{min}, and isotropic lines of sight. For NRSur7dq4: mass ratios 1–6 with either body first, total masses 20–300 $M_\odot$, sampling at 1–8 kHz, and f_{min} above the surrogate's start or zero.



Largest relative error of each draw against total mass. Left: IMRPhenomXPNR on 4,096 uniformly spaced frequencies from f_{min} to min(0.3/M, 2048 Hz), without the parity fix against LALSimulation 6.2.0, and with it against 6.2.0 patched with lalsuite !2593. Right: NRSur7dq4 against `SimInspiralChooseTDWaveform`. Data: `report/data/validation_*.json`.

NRSur7dq4 agrees with LAL to 2.8×10^{-12} at most (median 2.6×10^{-13}), with LAL's length and epoch in every compared draw (200/200). LAL evaluates all 200 draws, and so does waveformjax. The residual comes from summing the fits in a different order and from XLA's own `exp`, `sin` and `cos`. The rotation to the inertial frame and the splines are linear, so the errors stay at rounding level. waveformrs, which agrees with LAL bit for bit, gives the same comparison (2.8×10^{-12}).

IMRPhenomXPNR agrees with LAL 6.2.0 to 5.1×10^{-5} at most, with median 6.0×10^{-7}, a 90th percentile of 2.8×10^{-6} and mismatches below 2.1×10^{-9}. The tests show where the difference comes from. The (2,2) mode alone, with its PNR angles and antisymmetric part, agrees to ~1e-9. The higher modes agree to ~1e-5, as different builds of LAL agree with each other. LAL fixes the intermediate amplitude of each higher mode with a fourth-order finite difference of the inspiral amplitude, with a step of 1e-9, which turns rounding of 1e-16 into ~1e-7 in the derivative. It fixes the intermediate phase of the (3,2) mode with a second derivative of a phase of up to 1e4 rad, by central differences with a step of 1e-7, amplifying rounding by ~1e14. Without fused multiply-adds, which JAX does not expose portably, the JAX arithmetic rounds differently from LAL's arm64 build, and these finite differences turn that into a visible difference. Neither value is more accurate than the other. The errors grow with the total mass, where the merger and ringdown, and so the higher modes, carry more of the band: the median is 3.9×10^{-7} below 20 $M_\odot$ and 1.9×10^{-6} above 150 $M_\odot$. The largest errors, a few 1e-5, are for heavy binaries with in-plane spins near 0.9. 2% of the draws exceed 1e-5.

With the parity fix, the model agrees with LAL 6.2.0 patched with lalsuite !2593 to 4.7×10^{-5} (median 5.3×10^{-7}, mismatch $\leq 3.0\times10^{-9}$, over 193 draws). On the same draws waveformrs agrees with that patched library to 3.4×10^{-15}. LAL evaluates all 200 draws; waveformjax fails on 7 of them.

Conversions

FUNCTION	MODEL	BINARY	SETTING	MODES	SAMPLES	GRID	VS LAL	VS WAVEFORMRS	LAL MULTIBANDING	WAVEFORMJAX	WAVEFORMRS	LAL
`simulate_td`	IMRPhenomXPNR	36+29	4096 Hz from 20 Hz	all	13,006	exact	2.4×10^{-7}	2.4×10^{-7}	8.0×10^{-5}	38.42 ms	4.92 ms	12.52 ms
`simulate_td`	IMRPhenomXPNR	36+29	4096 Hz from 20 Hz	(2,±2)	13,006	exact	5.2×10^{-11}	5.2×10^{-11}	7.4×10^{-5}	25.77 ms	2.14 ms	6.04 ms
`simulate_td`	IMRPhenomXPNR	36+29	2048 Hz from 15 Hz	all	11,142	exact	2.4×10^{-7}	2.4×10^{-7}	4.2×10^{-5}	40.65 ms	8.05 ms	19.80 ms
`simulate_td`	IMRPhenomXPNR	36+29	2048 Hz from 15 Hz	(2,±2)	11,142	exact	8.6×10^{-12}	8.6×10^{-12}	4.0×10^{-5}	26.33 ms	3.37 ms	8.00 ms
`simulate_td`	IMRPhenomXPNR	60+40	2048 Hz from 20 Hz	all	5,398	exact	3.6×10^{-6}	3.6×10^{-6}	1.7×10^{-4}	31.22 ms	3.80 ms	9.28 ms
`simulate_td`	IMRPhenomXPNR	20+12	4096 Hz from 20 Hz	all	26,662	exact	2.6×10^{-6}	2.6×10^{-6}	3.9×10^{-5}	471 ms	20.64 ms	47.26 ms
`simulate_fd`	IMRPhenomXPNR	36+29	Δf from LAL, 20–2048 Hz	all	8,193	exact	2.9×10^{-8}	2.9×10^{-8}	3.8×10^{-5}	27.71 ms	3.82 ms	9.53 ms
`simulate_fd`	IMRPhenomXPNR	36+29	Δf from LAL, 20–2048 Hz	(2,±2)	8,193	exact	3.3×10^{-11}	3.3×10^{-11}	3.8×10^{-5}	23.87 ms	1.57 ms	3.67 ms
`simulate_fd`	IMRPhenomXPNR	36+29	Δf = 0.25, 19–1000 Hz	all	4,097	exact	2.8×10^{-8}	2.8×10^{-8}	3.6×10^{-5}	34.17 ms	4.01 ms	9.28 ms
`simulate_fd`	IMRPhenomXPNR	60+40	Δf = 0.125, 15–1024 Hz	all	8,193	exact	4.1×10^{-7}	4.1×10^{-7}	4.0×10^{-5}	34.13 ms	4.79 ms	11.41 ms
`simulate_td`	NRSur7dq4	40+20	4096 Hz from 20 Hz	all	4,225	exact	1.3×10^{-12}	1.3×10^{-12}	–	5.19 ms	1.29 ms	7.26 ms
`simulate_td`	NRSur7dq4	40+20	2048 Hz from 25 Hz	all	1,151	exact	3.1×10^{-13}	3.1×10^{-13}	–	4.82 ms	1.28 ms	6.89 ms
`simulate_td`	NRSur7dq4	70+30	2048 Hz from 14 Hz	all	2,619	exact	2.6×10^{-14}	2.6×10^{-14}	–	4.31 ms	1.34 ms	6.76 ms
`simulate_fd`	NRSur7dq4	40+20	Δf from LAL, 20–2048 Hz	all	4,097	exact	1.1×10^{-13}	1.1×10^{-13}	–	4.64 ms	1.20 ms	7.27 ms
`simulate_fd`	NRSur7dq4	40+20	Δf = 0.25, 19–1000 Hz	all	4,097	exact	8.5×10^{-14}	8.5×10^{-14}	–	4.81 ms	1.46 ms	7.32 ms
`simulate_fd`	NRSur7dq4	70+30	Δf = 0.125, 14–1024 Hz	all	8,193	exact	3.5×10^{-14}	3.5×10^{-14}	–	4.53 ms	1.20 ms	6.63 ms

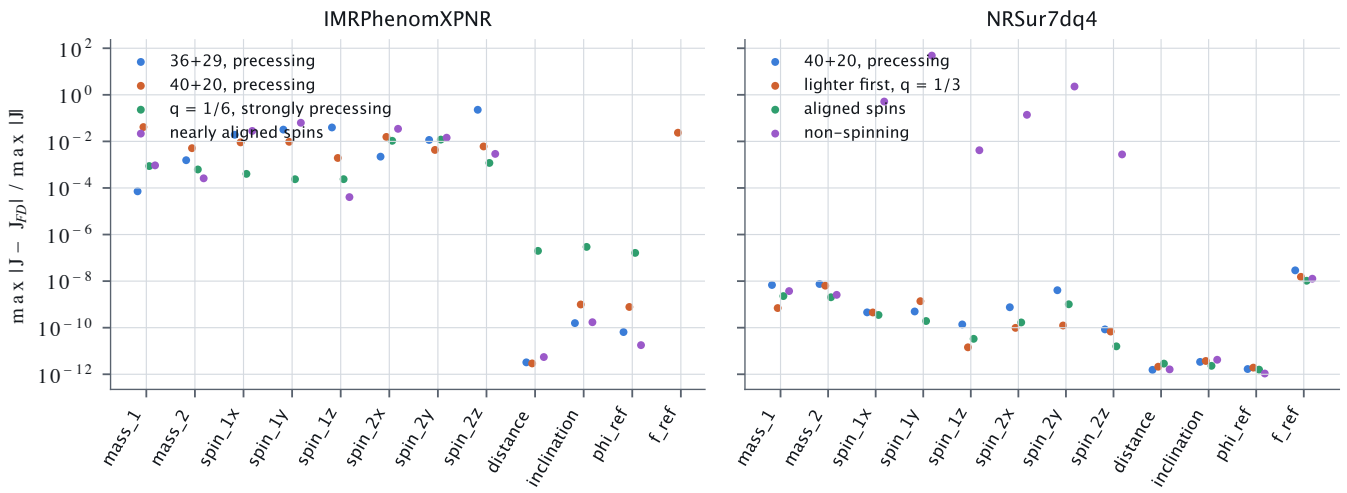
End-to-end `simulate_td` and `simulate_fd` against `SimInspiralTD` and `SimInspiralFD` with multibanding off. IMRPhenomXPNR is without the parity fix. "Grid" says whether the length, epoch and sampling are LAL's exactly. Errors are max $|h - h_{LAL}|$ / max $|h_{LAL}|$ over both polarizations. "LAL multibanding" is how far LAL's default output, with multibanding on, is from the output without it. Times are for one call after the first. Data: `report/data/validation_conversions.json`.

All lengths, epochs and samplings are LAL's (16/16). With LAL's own model in place of the JAX one, the conversions agree with LAL to ~1e-12 of the peak in the time domain and to rounding in the frequency domain (`test_conversions.py`). The time-domain high-pass filter, whose poles are close to z = 1, amplifies the rounding differences of the Fourier transforms, which are not FFTW's, by ~1e4. End to end, NRSur7dq4 agrees to 1.3×10^{-12}, IMRPhenomXPNR's (2,2) mode to 5.2×10^{-11}, and all its modes to 3.6×10^{-6}, as the models do. LALSimulation's default multibanding, which waveformjax does not implement, changes LAL's own output by 3.6×10^{-5}–1.7×10^{-4}. This is far more than the differences between the implementations.

8 Derivatives

For each model and four binaries, every column of the Jacobian (`jax.jacfwd`) is compared with central finite differences of the compiled model. The error of a column is max $|J_i - D_i|$ / max $|J_i|$ over the frequencies or times and both polarizations. The finite

difference is the better of a second-order difference with a step of 1e-6 and a fourth-order difference with a step of 1e-4, both relative for the masses, the distance and f_{ref}. The first is limited by rounding, the second by the model's discontinuities.



Disagreement of each column of the Jacobian with finite differences, per parameter and binary. Data: `report/data/gradients.json`.

MODEL	BINARY	MASSES, DISTANCE, ANGLES, F_{REF}	SPINS	WORST PARAMETER	NOT DIFFERENTIABLE	FINITE
IMRPhenomXPNR	36+29, precessing	1.6×10^{-3}	2.3×10^{-1}	`spin_2z`	–	yes
IMRPhenomXPNR	40+20, precessing	4.1×10^{-2}	1.6×10^{-2}	`mass_1`	–	yes
IMRPhenomXPNR	q = 1/6, strongly precessing	8.7×10^{-4}	1.2×10^{-2}	`spin_2y`	–	yes
IMRPhenomXPNR	nearly aligned spins	9.3×10^{-4}	6.3×10^{-2}	`spin_1y`	–	yes
NRSur7dq4	40+20, precessing	2.9×10^{-8}	4.1×10^{-9}	`f_ref`	–	yes
NRSur7dq4	lighter first, q = 1/3	1.5×10^{-8}	1.4×10^{-9}	`f_ref`	–	yes
NRSur7dq4	aligned spins	1.0×10^{-8}	1.0×10^{-9}	`f_ref`	–	yes
NRSur7dq4	non-spinning	1.3×10^{-8}	–	`f_ref`	`spin_1x`, `spin_1y`, `spin_1z`, `spin_2x`, `spin_2y`, `spin_2z` (zero spin)	yes
simulate_td IMRPhenomXPNR (2,2)	36+29, precessing	9.7×10^{-8}	2.5×10^{-7}	`spin_2x`	–	yes
simulate_fd NRSur7dq4	40+20, precessing	2.0×10^{-8}	3.9×10^{-9}	`f_ref`	–	yes

Largest column error by group of parameters. The conversions are `simulate_td_function` of IMRPhenomXPNR's (2,2) mode at 1024 Hz from 25 Hz and `simulate_fd_function` of NRSur7dq4 up to 1024 Hz from 20 Hz.

The derivatives are those of the function JAX computes, which is smooth almost everywhere but not everywhere. Wherever LAL makes a discrete choice that depends on the binary, the waveform is continuous only up to the size of that choice's effect, and the derivative is that of the branch taken. Such choices include the number of SpinTaylor samples, where LAL's adaptive integration stops, the node where NRSur7dq4's integration starts, the precession prescription, and the order of the intermediate polynomials of the higher modes. In IMRPhenomXPNR the SpinTaylor waveform jumps by about LAL's integration error, ~1e-9 relative, wherever the number of samples changes. A finite difference that straddles such a jump is wrong by the jump divided by the step, which is why small steps are used. The spins are the parameters that move the precession most, and they show the largest disagreements. For nearly aligned in-plane spins the waveform is also far from linear on the scale of the step. None of this affects the derivatives themselves, which are those of the smooth pieces.

One case is not a jump but a point where no derivative exists. After every step of its dynamics, NRSur7dq4 rescales each spin to its initial magnitude (LAL's `PrecessingNRSur_normalize_y`). For a body with no spin, an infinitesimal spin then evolves by a factor that depends on its direction, not only on its size. The waveform therefore has a cone-shaped kink in that body's three spin components: one-sided finite differences tend to different values, and no step size makes them converge. At such a point the JAX derivative is finite, as the transform tests require, but it is a convention rather than a derivative. The table reports these components apart. As soon as a body has any spin, even an aligned one, its derivatives agree with finite differences as for the other binaries.

IMRPhenomXPNR: under investigation. For the full IMRPhenomXPNR waveform, the finite differences disagree with the derivatives by up to 2.3×10^{-1}, by far the most in the in-plane spins. The transform tests, which compare the gradient of a weighted sum of the (2,2) mode with finite differences, agree to ~1e-7. The likely cause is in the higher modes. Their intermediate region is fixed by LAL's finite differences with steps of 1e-9 and 1e-7, which leave rounding noise of ~1e-8 in the waveform. That noise is not smooth in the parameters, so a parameter step of 1e-6 sees a slope of up to ~1e-2 relative from it alone, while the JAX tangents are

those of the exact derivatives. If so, the derivatives are right and the finite differences are not. The check still to run: the (2,2) mode alone, and the full model with larger steps, where that noise matters less.

Cost of derivatives

FUNCTION	COMPILE	VALUE	JVP	COMPILE JACFWD	JACFWD	COMPILE GRAD	GRAD	COMPILE VMAP(GRAD)	VMAP(GRAD) / BINARY	GRAD VS J^TW
IMRPhenomXPNR	14.7 s	10.17 ms	94.16 ms	47.1 s	93.86 ms	47.6 s	96.53 ms	72.1 s	88.79 ms	3.1×10^{-15}
NRSur7dq4	2.5 s	2.75 ms	6.18 ms	5.3 s	19.36 ms	8.8 s	18.70 ms	11.0 s	13.42 ms	5.9×10^{-15}
simulate_td IMRPhenomXPNR (2,2)	8.2 s	8.77 ms	76.37 ms	28.5 s	99.80 ms	28.3 s	78.29 ms	46.7 s	83.60 ms	3.2×10^{-13}
simulate_fd NRSur7dq4	1.7 s	2.65 ms	6.02 ms	5.4 s	20.13 ms	8.9 s	18.99 ms	10.0 s	16.06 ms	6.3×10^{-15}

Times of the first call (which compiles) and of later calls, for IMRPhenomXPNR on 2048 frequencies, NRSur7dq4 on 2048 samples and the two conversions. `grad` is of a scalar, a weighted sum of the polarizations. `vmap(grad)` is over 8 binaries (4 for the conversions). The last column is the relative difference between `jax.grad` and the Jacobian contracted with the same weights.

For IMRPhenomXPNR a gradient costs 96.53 ms, 9.5× the cost of a waveform. It is a forward-mode Jacobian of twelve columns, evaluated together. Its compilation takes 47.6 s. A `jax.jvp` along a direction that moves every parameter costs as much as `jax.jacfwd`, because the rule computes the columns of all the parameters with non-zero tangents and then contracts them. When only some parameters are differentiated, as in `jax.grad` with respect to the masses alone with the rest of the binary held fixed, only their columns are computed. NRSur7dq4 is differentiated directly in reverse mode: a gradient costs 18.70 ms (6.8× a waveform) after 8.8 s of compilation.

9 Performance

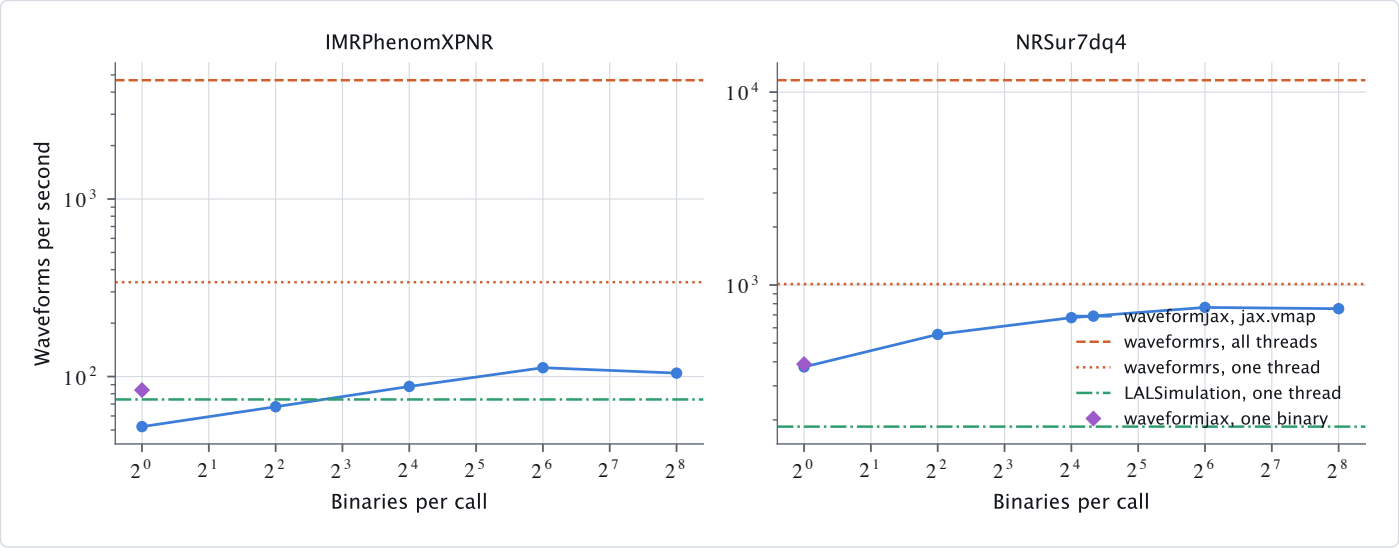
All timings are on the CPU (Apple M2 Ultra, 24 cores), with JAX's default settings and LALSimulation single-threaded, except those of NRSur7dq4 in MLX (section 4). The binaries and grids are those of waveformrs's Python benchmark: IMRPhenomXPNR for 36 + 29 $M_\odot$ on 8033 frequencies (20–1024 Hz, $\Delta f = 1/8$ Hz), and NRSur7dq4 for 40 + 20 $M_\odot$ at 4096 Hz from 20 Hz. IMRPhenomXPNR is timed without the parity fix, which costs the same.

WAVEFORM	FIRST CALL	PUBLIC FUNCTION	COMPILED FUNCTION	WAVEFORMRS	LAL	LAL / COMPILED
IMRPhenomXPNR, 8033 frequencies	17.7 s	17.22 ms	11.92 ms	3.01 ms	13.97 ms	1.2×
NRSur7dq4, 4096 Hz from 20 Hz	2.4 s	2.90 ms	2.57 ms	1.03 ms	5.39 ms	2.1×
`simulate_td` of IMRPhenomXPNR, 4096 Hz from 20 Hz	16.0 s	25.96 ms	–	5.38 ms	12.53 ms	0.5×
`simulate_fd` of NRSur7dq4, up to 2048 Hz from 20 Hz	97.27 ms	3.20 ms	–	1.08 ms	5.41 ms	1.7×

One waveform per call. "First call" includes LAL's checks, tracing and compilation. "Public function" is a later call of `wf.polarizations` (or `td_polarizations`, `simulate_*`), including the checks on the host and the copy of the result. "Compiled function" is the jitted pure function alone. waveformrs is its fastest path from Python. Medians of 7 repeats. Every call is for a different binary of a cycle of 16. Data: `report/data/timings.json`.

A compiled IMRPhenomXPNR waveform takes 11.92 ms, against 13.97 ms for LALSimulation and 6.31 ms (3.01 ms on its grid path) for waveformrs. The public function adds 4.86 ms of checks on the host. To decide, as LAL would, whether a binary must change its precession prescription, they run a compiled pass through the set-up and the SpinTaylor integration; the pure function skips them. The JAX version is 4.0× slower than waveformrs's fastest path. The difference is the cost of being traceable. Every `jnp.where` evaluates both branches. Loops whose length depends on the binary run as `while_loop`s, which XLA cannot fuse across iterations. The SpinTaylor replica must step one GSL step at a time, and XLA's CPU code has no fused multiply-adds. XLA keeps about 3.3 cores busy for one IMRPhenomXPNR waveform, while LAL and waveformrs use one; JAX on a single core is still to be measured. NRSur7dq4 is dominated by vectorisable work (the fits at the nodes and the rotation of every mode sample), and takes 2.57 ms against 5.39 ms for LAL and 1.03 ms for waveformrs. The conversion `simulate_td` of IMRPhenomXPNR takes 25.96 ms, against 12.53 ms for LAL with multibanding off (11.47 ms with its default multibanding) and 5.38 ms for waveformrs.

Batches



Waveforms per second against the number of binaries per call of `jax.jit(jax.vmap(model))`, for binaries around the benchmark binary (masses ±10%, spin components in ±0.5, random angles). The horizontal lines are waveformrs's batch function and a loop over LALSimulation for 64 binaries.

BINARIES PER CALL	1	4	16	64	256
IMRPhenomXPNR, waveforms/s	52	68	88	112	105
IMRPhenomXPNR, compilation	20.7 s	21.7 s	22.0 s	22.7 s	24.8 s
IMRPhenomXPNR, cores busy	3.4	4.1	5.5	5.6	5.3
NRSur7dq4, waveforms/s	376	554	677	765	754
NRSur7dq4, compilation	2.0 s	2.0 s	2.0 s	2.1 s	2.4 s
NRSur7dq4, cores busy	5.8	4.2	4.4	7.0	7.1

Throughput, compilation time and cores kept busy (process CPU time over wall time) for each batch size. NRSur7dq4 is evaluated on LAL's output grid of the benchmark binary.

64 BINARIES	LAL LOOP	WAVEFORMRS, 1 THREAD	WAVEFORMRS, ALL THREADS	WAVEFORMJAX	FIRST CALL
IMRPhenomXPNR	74	340	4,667	70	23.8 s
NRSur7dq4	184	1,011	11,524	734	2.4 s

Waveforms per second through the public batch functions (`batch_polarizations`, `batch_td_polarizations`), including LAL's checks of every binary on the host.

Under `jax.vmap`, IMRPhenomXPNR reaches 112 waveforms per second. Batching amortises the per-call overhead and XLA spreads the vectorised work over cores, but the loops run to the longest binary in the batch and the branches of all binaries are computed. Each batch size is compiled separately (22.7 s for 64). waveformrs on all threads, with a thread per binary and no padding, remains faster on a CPU. The JAX version is meant for code that is itself written in JAX: a likelihood that is jitted and differentiated end to end, or a sampler that runs on an accelerator.

10 Differences from LALSimulation

- **Rounding, not bit-identity.** waveformrs reproduces LAL bit for bit on macOS arm64, using the fused multiply-adds that clang emits. JAX has no portable fused multiply-add and XLA reorders arithmetic, so waveformjax agrees with LAL to rounding (NRSur7dq4, the (2,2) mode of IMRPhenomXPNR) or to the rounding amplified by LAL's own finite differences (the higher modes, ~1e-5). Only on the host, where the conversions' lengths and epochs hinge on the last bits, are LAL's fused multiply-adds reproduced exactly. The inflection points of the PNR `beta`, roots of a strongly cancelling cubic, agree with LAL's to ~1e-9 (relative), which moves its connection frequencies by as much: the waveforms change by a mismatch of ~1e-16.
- **SpinTaylor backward integration.** The integration from `f_ref` down to `f_min` is a dense Dormand–Prince solution, not LAL's RKF45 steps. It differs from LAL's samples by LAL's integration error (~1e-9).
- **The zero crossings of the quintic intermediate amplitude.** LAL's `CrossZeroP5`, which decides whether a higher mode's intermediate amplitude drops from a fifth-order polynomial, reads shifted entries of GSL's packed roots. It misses some roots and depends on the order in which GSL finds them, which rounding can change. Here every root is tested with its own real and imaginary parts (`hm/roots.py`), so the order does not matter and GSL's QR steps can work on whole rows and columns. The quick tests are unchanged (IMRPhenomXPNR against LAL: relative error 3.5e-7, mismatch 4.5e-14; the extended (2,2) mode:

mismatch 2.9e-8, as before). Of 200 random binaries, none changes with the default options and one changes with the extended angles (mismatch 5.8e-8). Among the single-mode tests, one binary takes another polynomial for its (3,−3) mode, which then differs from LAL by a mismatch of 4.4e-7.

- **Values where LAL divides by zero.** At isolated points of the parameter space, LAL's MSA formulas divide by an exact zero and produce inf or NaN. Here the values there are finite, and so are the gradients nearby, for example at equal masses or for a single spin.
- **No multibanding.** LAL's `SimInspiralChooseFDWaveform` uses multibanding by default for IMRPhenomXPNR; waveformjax, like waveformrs, evaluates every frequency. `SimInspiralChooseFDWaveformSequence` has multibanding off, and the conversions are compared with it off.
- **No data race.** LAL's MSA and PNR angles race under OpenMP. waveformjax agrees with single-threaded LAL.
- **The scale of the output.** Both models return the strain times 10^{21}, and compute the amplitude without SI intermediates (NRSur7dq4 the masses too), which changes their float64 values by rounding. `scaled_strain=False`, which `WaveformOptions.lalsimulation_defaults()` also sets, gives LAL's scale.
- **Two options that are not LAL's defaults.** With `version_fallback=False` the compiled model contains only the SpinTaylor prescription and is NaN where LAL would switch to the MSA or NNLO angles. With `extended_spintaylor=True` it is a different model: the SpinTaylor angles for every binary, with LAL's discrete switches made continuous. A separate report, `report/extended`, maps where the SpinTaylor angles fail and describes and validates the extended model.

11 Limitations and future work

- **Two approximants.** Only IMRPhenomXPNR and NRSur7dq4 are public. IMRPhenomXAS and the 122019 higher modes exist as parts of IMRPhenomXPNR, but IMRPhenomXHM, XP and XPHM with their own flags, TaylorF2 and IMRPhenomD are not exposed.
- **Compilation time.** IMRPhenomXPNR takes 15–20 s to compile, its gradient under a minute, and every array shape and batch size compiles separately. JAX's persistent compilation cache (`jax_compilation_cache_dir`) makes the cost one-off per machine. The tests use it; the package does not set it up.
- **Accelerators.** The JAX versions ran on the CPU only. In JAX, both models run in float32 only through their pure functions, since the public functions and the conversions are float64; IMRPhenomXPNR's default model is accurate only in float64, which is slow on consumer GPUs. Neither model has been measured on a GPU through JAX. NRSur7dq4 and the lean IMRPhenomXPNR run on Apple GPUs through their MLX versions (section 4, section 3), which have limits of their own:
  - it is float32 only, since MLX has no float64 on the GPU;
  - they cannot be differentiated in forward mode (`mx.jvp`) or mapped with `mx.vmap`, which MLX does not carry through custom Metal kernels. They take batches themselves; NRSur7dq4's gradients are reverse mode, IMRPhenomXPNR's are its forward-mode Jacobian contracted in a custom VJP;
  - IMRPhenomXPNR in MLX has only the pure function at given frequencies (no LAL grids or conversions), depends on JAX to trace the model, and sizes its arrays with the binaries (the SpinTaylor steps, the grid of the PNR interpolants), so it reads sizes back from the GPU and does not run under `mx.compile`.
- **Forward-mode derivatives of IMRPhenomXPNR.** A gradient costs one Jacobian column per parameter with a non-zero tangent, evaluated together. That is affordable for the twelve parameters of a binary (section 8), but it does not scale to derivatives with respect to the frequencies or to many other inputs. A reverse-mode path would need the loops with traced lengths rewritten with fixed bounds.
- **Discontinuities.** The waveform inherits LAL's discrete choices. Gradient-based samplers near a regime boundary, such as in-plane spins near 1e-6, or a SpinTaylor integration that just fails, see the derivative of one side only.
- **Long waveforms.** For light binaries from low frequencies the SpinTaylor splines have millions of knots and the conversions' Fourier transforms millions of samples. These are fixed-shape arrays held in memory, and both cost and compilation grow with them.
- **Multibanding.** LAL's multibanding is not implemented (as in waveformrs).

12 Reproducing the results

```
# environment (conda-forge), with waveformrs checked out next to waveformjax
mamba create -n waveformjax -c conda-forge python=3.12 python-lalsimulation jax numpy h5py scipy matplotlib pytest
conda activate waveformjax
pip install -e .
(cd ../waveformrs/python && maturin develop --release)      # optional: the waveformrs comparisons
pip install -e ".[mlx]"                                        # optional: NRSur7dq4 and IMRPhenomXPNR in MLX (Apple silicon)

# the tests (the data file NRSur7dq4_v1.0.h5 from zenodo.org/records/14999310,
# found through NRSUR7DQ4_DATA, LAL_DATA_PATH or ../waveformrs/lal_data)
WAVEFORMJAX_SLOW=1 python -m pytest

# IMRPhenomXPNR with the parity fix needs LALSimulation 6.2.0 + lalsuite !2593
CC=/path/to/conda-forge/clang-19 bash ../waveformrs/scripts/build_patched_lalsimulation.sh ../lalsuite build/lal-2593
export PATCHED_LALSIMULATION=$PWD/build/lal-2593/liblalsimulation.37.dylib

# every measurement and this report, on an idle machine (MLX_PYTHON: a Python
# with MLX, if not the same)
bash report/measure_all.sh

# or one at a time
python report/validate_xpnr.py --n-draws 200
python report/validate_nrsur7dq4.py --n-draws 200
python report/float32_nrsur7dq4.py
python report/mlx_nrsur7dq4.py                              # needs MLX
python report/mlx_xpnr.py                                   # needs MLX
python -m pytest tests/test_nrsur7dq4_mlx.py tests/test_xpnr_mlx.py --junitxml=report/data/pytest_mlx.xml
python report/validate_conversions.py
python report/gradients.py
python report/benchmark.py
python report/build_report.py
```

13 Commit history

The commits on `main`, oldest first. The package was built model by model and milestone by milestone, each checked against LAL before the next. The conversions and the speed-ups of IMRPhenomXPNR were developed in parallel on two branches and merged. Commits that only add or regenerate this report are not listed. The repository has no remote.

COMMIT	TYPE	SUMMARY
229d095	FEAT	the waveformjax package: the waveformrs interface over JAX models
7494344	DOC	per-model transform tests in AGENTS.md
48c6d70	FEAT	NRSur7dq4 data reader, parameter-space fits and GSL interpolation in JAX
839cf04	FEAT	NRSur7dq4 dynamics as fixed-size JAX loops
4da7038	FEAT	NRSur7dq4 coorbital waveform, frame rotation and polarizations
cf2462d	FEAT	NRSur7dq4 MODEL: LAL's checks, output grid, modes and batches
4b1fead	TST	find the NRSur7dq4 data next to a parent directory (git worktrees)
c296de4	TST	NRSur7dq4 against LALSimulation 6.2.0 and under jit, vmap and grad
20502f6	PERF	NRSur7dq4 splines with GSL's factorised systems, cached output grids
a303ecb	DOC	NRSur7dq4 in the README and its original code in NOTICE.md
9d7cb40	DOC	NRSur7dq4 dynamics docstring; share |ra|^2a |rb|^2b in the rotation
f5f9745	PERF	NRSur7dq4 dynamics steps as a few vector operations
d0acb33	MAINT	drop an unused table of the NRSur7dq4 waveform module
0c084c5	FEAT	IMRPhenomXAS (2,2) mode in JAX, with the fits generated from waveformrs
ea84f6e	FEAT	PNR fits and angle helpers, and the SpinTaylor evolution in JAX
91fe045	FEAT	IMRPhenomXHM release 122019 higher modes in JAX
2fcc136	FEAT	MSA (223), NNLO (102) and SpinTaylor (330) precession angles in JAX
8761b4d	FEAT	IMRPhenomXPNR polarizations in JAX
c7f6cba	FIX	one optimization barrier per value
59e311b	FEAT	IMRPhenomXPNR J-frame modes, validation and transform tests
0b05193	TEST	one switch, WAVEFORMJAX_SLOW=1, for the random-draw tests of both models
afe668f	TEST	the NRSur7dq4 random-draw test requires most draws compared, for any NRSUR7DQ4_DRAWS

COMMIT	TYPE	SUMMARY
f57a940	FEAT	LAL's conditioning in JAX: duration bounds, epochs, tapers and the Butterworth filter
787c98e	FEAT	simulate_td and simulate_fd: LAL's SimInspiralTD and SimInspiralFD
a182f49	DOC	the conversions in the README, and their original code in NOTICE.md
4e5a7ae	TEST	the conversions against SimInspiralTD and SimInspiralFD
c9e8eba	DOC	the XPNR adapter's values depend on the whole frequency array
9bdcf41	DOC	the accuracy of the conversions, as measured
568227a	TEST	the conversions as pure functions under jit, vmap and grad
545046e	PERF	SpinTaylor derivatives with the 3-vectors as arrays
eb49f8b	PERF	GSL's quintic solver in loops that end where GSL's do
4fe9828	PERF	IMRPhenomXPNR derivatives in forward mode, batched under jax.vmap
a2d6a68	PERF	the intermediate polynomials of the higher modes chosen together
8e042ff	PERF	one branch for PNR versions 102 and 223; smaller SpinTaylor code
b5c0df1	PERF	only the PNR beta of the binary's region; alpha of the antisymmetric part where used
864c255	PERF	batch-aware branches and loop bodies traced under jax.jit
4476314		Merge the IMRPhenomXPNR compile, batching and forward-mode speed-ups
deeffce	FIX	the conversions of IMRPhenomXPNR differentiate the model in forward mode
0a93b55	FIX	NRSur7dq4 data read inside jax.jit are concrete arrays
027e282	FEAT	IMRPhenomXPNR without fallbacks, and extended, continuous SpinTaylor angles
c31b9c3	FEAT	NRSur7dq4 in float32, with the strain times 1e21 by default
366535e	FEAT	smoother extended XPNR angles where LAL samples a step or a branch
321ebea	DOC	the smoothness study of the extended XPNR model (measurement and draft section)
26c400e	FIX	finite gradients of the extended XPNR at aligned and vanishing spins
aa32207	FEAT	NRSur7dq4 in MLX (float32, Apple GPUs), as a separate module
157fd2e	FIX	the smooth XPNR angles where L is close to -J
d78a40c	FEAT	the MLX NRSur7dq4 dynamics in one Metal kernel, with its Jacobian
7632c48	PERF	the MLX NRSur7dq4 rotation to the inertial frame in Metal kernels
a315b13	PERF	tune the MLX NRSur7dq4 dynamics kernel
87c01bb	FEAT	the MLX NRSur7dq4 on LAL's grid, in batches and conditioned, with its report section
aa460c3	FEAT	IMRPhenomXPNR compiles only the SpinTaylor angles by default, direct extended angles, quick tests
6994cac	PERF	evaluate the SpinTaylor splines of the XPNR set-up at all its points at once
630477e	PERF	one copy of the SpinTaylor derivatives per RKF45 step, materialized mode arrays
3dfe8cf	PERF	the PNR beta variants of the interpolation grid computed once each
4c32137	PERF	the carry of the XPNR set-up scan starts dependent on the binary
4008e75	PERF	h_+ and h_x of a twisted-up mode computed in one kernel
5f97d83	FIX	CrossZeroP5 tests every root of the quintic; QR steps on whole rows
eeec783	PERF	the double QR sweep of the quintic's roots in a loop over k
a6ce002	PERF	plain fused multiply-adds in the PNR beta connection
1df3534	PERF	the intermediate polynomials of the higher modes from a linear solve
6123358	PERF	the (2,2) coefficients of the antisymmetric mode reused by the symmetric one
a87a5c0	PERF	the SpinTaylor alpha of the antisymmetric mode evaluated with the PNR grid
471262c	PERF	the coefficients of the (2,1), (3,3) and (4,4) modes in one vmapped body
12eef25	PERF	the (3,2) mode in the vmapped body of the higher modes
a2994a6	PERF	batched branches merge into one output rather than selecting among zeros
1aaa72e	PERF	SpinTaylor loop bodies small enough to run on XLA's calling thread
3e07c6a	PERF	the antisymmetric alpha of the grid loop only below its transition

COMMIT	TYPE	SUMMARY
d253bf0	TST	JAX's persistent compilation cache for the tests
09c88fd	PERF	the cross products of the SpinTaylor equations from gathers
ac3b193	PERF	the moving average of the antisymmetric amplitude ratio in one solve
3ff80dc	PERF	one copy of the stopping condition in the smooth end of the fine run
2b1e132	BUG	finite derivatives of the evolved spins at aligned spins
59cc4cc		ENH: IMRPhenomXPNR in float32, and the strain times 1e21
f634a68		ENH: the lean IMRPhenomXPNR (WaveformOptions(lean=True))
3c42b28	PERF	lean IMRPhenomXPNR in one pass, beta connection evaluated once
7d871df	PERF	lean quintic zero-crossing test by signs; exact HM derivatives at one point
38a70a3	DOC	the lean IMRPhenomXPNR in the report
b7b6028	TST	the lean IMRPhenomXPNR in stages, the reference of its MLX port
90372b6		ENH: jaxprs of the lean model's stages as scalar programs and Metal kernels
78c4b0e		ENH: the set-up of the lean IMRPhenomXPNR as generated MLX kernels
7183d19		ENH: the lean SpinTaylor solution as Metal kernels
bdb42c9		ENH: the SpinTaylor angles at points and the PNR interpolants in MLX
5fdeedb		ENH: the lean IMRPhenomXPNR in MLX: polarizations, Jacobian and mx.grad
07994df	BUG	finite float32 derivatives of the IMRPhenomXPNR higher modes
d74e287	PERF	the MLX lean IMRPhenomXPNR: SIMD lanes per component, packed stages
fe9389f	PERF	the MLX lean IMRPhenomXPNR: tables read by index, batches in chunks